

Distributed APF-Based Control for Perpetual First-Arrival Target Reaching by a Quadcopter Swarm.

1st Daniel Grønhaug
Department of Mathematics
University of Oslo
Oslo, Norway
dfgronha@math.uio.no

2nd Oliver Ekeberg
Department of Mathematics
University of Oslo
Oslo, Norway
oliveroe@math.uio.no

Abstract—This paper presents a decentralized control strategy for a multi-agent UAV swarm engaged in a sequential target-reaching mission. The approach employs an artificial potential field based controller, which generates velocity commands for each agent through the superposition of attractive forces toward the target and repulsive forces from other agents. The UAVs vie to be the first to enter a defined cutoff radius around the stationary target. The successful agent then completes the approach, after which a new target is generated and the cycle repeats. The strategy is evaluated in simulation based on the speed of target acquisition and guaranteed collision avoidance. To ensure reproducibility, all simulation code and a containerized deployment environment are provided in an accompanying GitHub repository. The results from these simulations are presented and discussed.

Index Terms—Artificial potential fields, multi-agent systems, drone swarms, target-reaching, ROS2, Webots

I. INTRODUCTION

The cooperative control of multi-agent systems is a cornerstone of swarm robotics, enabling applications such as surveillance, exploration, transport, and search [1]. Within this domain, the Target Reaching Problem (TRP) addresses the challenge of navigating single or multiple agents from a starting point to a specific destination. This work focuses on a sequential TRP where the target is stationary and above ground.

To coordinate the swarm, a decentralized control strategy based on Artificial Potential Fields (APF) is adopted. This method generates a guidance vector for each UAV by superimposing an attractive potential toward the target with repulsive potentials from every other agent [2]. Afterwards, the vector is saturated. The resulting vector is interpreted as a velocity command, which serves as the reference input to a lower-level motor controller. In an empty world, these commands produce trajectories that lead the agents toward the target. To prevent crowding at the target, the agents vie to be the first to enter a specified cutoff radius around the target. The winning agent proceeds to complete the approach, while the others disengage their attraction to the current target.

Once the target is reached, a new one is generated, and the cycle repeats, forming a continuous sequential mission.

The proposed control algorithm was implemented and evaluated in simulation using the physics-based robot simulator Webots [3] and the Robot Operating System 2 (ROS 2) middleware [4]. The UAV models are DJI Mavic 2 PRO quadcopters (Mavics), imported along with their motor controllers via the Webots ROS 2 interface [5]. All simulation code is publicly available in a GitHub repository [6]. The simulation environment was adapted from [7] and [8].

The results from the simulation show that all drones successfully reaches the first target without colliding. After the first episode, drone three turns inactive, while drone one and two continue to reach successive targets, until drone two also becomes inactive after episode five. An inactive drone corresponds to a loss of flight stability, as a result of poor low-level stability, and not agent collisions. Throughout the simulations, no collisions between drones are observed, indicating that the repulsive component of the control law maintains safe separation.

II. PROBLEM FORMULATION

This section formalizes the sequential Target Reaching Problem (TRP) for a three-agent quadcopter swarm in an idealized simulation environment. It details the system including agent models, velocity-level control architecture, and randomized target generation and specifies the control objectives of target acquisition and collision avoidance under perfect global information and decentralized computation.

A. Scenario description

The system comprises a swarm of three autonomous DJI Mavic 2 PRO quadcopters operating in a three-dimensional Euclidean space, $\mathbb{R}^3$. The simulated environment is an obstacle-free arena containing only a flat ground plane. The physics are governed by the default Webots rigid-body dynamics, supplemented with a global damping coefficient

of 0.5 to ensure numerical stability, creating an idealized, low-disturbance simulation.

At the start of a simulation, each quadcopter $i \in \{1, 2, 3\}$ is initialized at a fixed position $\mathbf{p}_i(0) \in \mathbb{R}^3$ with zero initial velocity, $\mathbf{v}_i(0) = \mathbf{0}$. The collective objective is to reach a stationary target point $\mathbf{T} \in \mathbb{R}^3$, defined as entering a spherical capture radius of 0.8 meters centered on $\mathbf{T}$. Once captured, the target respawns at a new location, repeating the process indefinitely.

The target coordinates $\mathbf{T} = (T_x, T_y, T_z)^\top$ are drawn from uniform distributions upon initialization and after each capture:

$$T_x, T_y \sim \mathcal{U}(-10, 10), \quad T_z \sim \mathcal{U}(3, 5) \quad (1)$$

B. Agent Representation and Control Level

Each agent is modeled as a DJI Mavic 2 PRO quadcopter within the simulation. Control is performed at the velocity-command level, whereby a high-level controller specifies desired body-frame velocities. The low-level dynamics including attitude stabilization, motor mixing, and thrust generation are managed by an embedded flight controller, the details of which are abstracted in this work. Consequently, our proposed controller generates velocity references rather than direct motor or thrust commands.

The resulting high-level control input for agent i is a four-dimensional vector:

$$\mathbf{u}_i(t) = (v_{x,i}(t), v_{y,i}(t), v_{z,i}(t), \dot{\psi}_i(t))^\top,$$

where $v_{x,i}$, $v_{y,i}$, and $v_{z,i}$ are the commanded linear velocities in the body frame, and $\dot{\psi}_i$ is the commanded yaw rate.

C. Control Objectives and Constraints

The primary control objective is to drive the swarm through a sequence of target positions while ensuring collision avoidance. This translates to minimizing the Euclidean distance between each agent and the current target, subject to a minimum safe separation distance between all agents.

The simulation employs idealized quadcopter dynamics, abstracting away numerous real-world constraints. These include energy consumption, actuator saturation, mechanical failure, and detailed aerodynamic effects such as turbulence. Communication is assumed to be perfect, meaning that factors like varying signal quality or delays between agents are not modeled.

The main practical constraint stems from the admissible input space for the velocity-level controller. As specific input limits for the embedded flight controller are not formally documented, conservative saturation limits for linear velocity and yaw rate were established empirically to achieve stable simulation behavior.

D. Information Structure and Decentralization

Each agent has continuous access to its own position and velocity vector, the current target position, and the positions of all other agents. This corresponds to a scenario with strong GPS signals (low noise) and a fully connected communication network.

The APF-based controller uses this information in a decentralized manner. Each agent computes its own velocity command via onboard processing, using its current state (position and velocity), the target position, and the positions of the other agents.

III. ARTIFICIAL POTENTIAL FIELD CONTROLLER

The control strategy builds upon the Artificial Potential Field (APF) approach originally developed by Khatib [9], augmented with two principal modifications: a winner-takes-all rule for target capture and explicit saturation of the resulting velocity commands. The linear velocity commands are generated from this modified APF, while a proportional controller governs the angular velocity, aligning the agent with its linear velocity command. We begin by presenting the core APF formulation.

A. Equations

Let $\mathbf{T}(t)$ denote the target position and $\mathbf{p}_i(t)$ the position of drone i , both defined in a global reference frame. The unsaturated desired velocity for drone i is the sum of an attractive component toward the target and repulsive components from all other agents:

$$\mathbf{v}_i^{\text{u-des}}(t) = \mathbf{a}_i(t) + \sum_{j \neq i} \mathbf{r}_{i,j}(t). \quad (2)$$

Attractive Potential. The attractive velocity is proportional to the vector pointing from the drone to the target:

$$\mathbf{a}_i(t) = \zeta (\mathbf{T}(t) - \mathbf{p}_i(t)), \quad \zeta > 0, \quad (3)$$

where ζ is a positive gain, which we set $\zeta = 0.5$. This original formulation can lead to crowding as all agents converge simultaneously. To enforce a clear capture sequence, the attraction is modified by a competitive rule: only the first agent to enter a capture radius q retains attraction to the target. The remaining agents have their attractive term deactivated. We set $q = 1$.

Formally, let $\mathcal{W}(t) \subset \{1, 2, 3\}$ denote the set of the winning agent at time t . Initially, $\mathcal{W}(0) = \emptyset$. The set is updated according to the following rules:

- 1) If $\mathcal{W}(t) = \emptyset$ and an agent i satisfies $\|\mathbf{T}(t) - \mathbf{p}_i(t)\|_2 \leq q$, then $\mathcal{W}(t^+) = \{i\}$. This agent is designated the winner.
- 2) If $\mathcal{W}(t) = \{i\}$ and the winning agent captures the target (i.e., $\|\mathbf{T}(t) - \mathbf{p}_i(t)\|_2 \leq 0.8$), then $\mathcal{W}(t^+) = \emptyset$ after the target respawns, resetting the competition for the new target.

When $\mathcal{W}(t) \neq \emptyset$, only the winning agent $i \in \mathcal{W}(t)$ maintains an attractive potential toward the target; all other agents

have their attractive term set to zero. The modified attractive potential becomes:

$$\mathbf{a}_i(t) = \begin{cases} \zeta (\mathbf{T}(t) - \mathbf{p}_i(t)), & \text{if } i \in \mathcal{W}(t) \text{ or } \mathcal{W}(t) = \emptyset, \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

This mechanism ensures that once an agent is within the capture radius, the others cease approaching the current target, preventing congestion.

Repulsive Potential. The repulsive velocity between agents i and j depends on their distance, $d_{i,j}(t) := \|\mathbf{p}_i(t) - \mathbf{p}_j(t)\|_2$, and is active only within a safety radius d :

$$\mathbf{q}_{i,j}(t) := \mathbf{p}_i(t) - \mathbf{p}_j(t) \quad (5)$$

$$\mathbf{d}_{i,j}(t) := d_{i,j}(t)^{-1} - d^{-1} \quad (6)$$

$$\mathbf{r}_{i,j}(t) = \begin{cases} \eta \mathbf{d}_{i,j}(t) d_{i,j}(t)^{-3} \mathbf{q}_{i,j}(t), & \text{if } d_{i,j}(t) \leq d \\ 0, & \text{otherwise} \end{cases} \quad (7)$$

where $\eta > 0$ is a repulsive gain and d defines the range of the repulsive interaction. We set $\eta = 2$ and $d = 1$.

Saturation. The computed velocity $\mathbf{v}_i^{\text{u-des}}(t)$ must be saturated to ensure stable flight. The following component-wise constraints are enforced by rescaling the vector when necessary:

$$|(\mathbf{v}_i^{\text{u-des}}(t))_z| \leq 1 \quad (8)$$

$$\sqrt{(\mathbf{v}_i^{\text{u-des}}(t))_x^2 + (\mathbf{v}_i^{\text{u-des}}(t))_y^2} \leq 1 \quad (9)$$

Note that with $(\mathbf{v}_i^{\text{u-des}}(t))_x$ we refer to the x component of $\mathbf{v}_i^{\text{u-des}}(t)$, likewise for y and z . Thus, the vertical velocity component is bounded by 1 m/s in magnitude, while the horizontal velocity in the xy -plane is similarly bounded by 1 m/s.

Let $\mathbf{v}_i^{\text{u-des-1}}(t)$ be the velocity vector after enforcing these saturation constraints, and let $\mathbf{v}_i^{\text{true}}(t)$ denote the current linear velocity of agent i in the world frame. To prevent abrupt changes that could destabilize the system, we impose a rate limit on the commanded velocity change. Specifically, the difference between the desired and actual velocities is constrained by:

$$d\mathbf{v}_i(t) := \mathbf{v}_i^{\text{u-des-1}}(t) - \mathbf{v}_i^{\text{true}}(t), \quad (10)$$

with the requirement that $\|d\mathbf{v}_i(t)\|_2 \leq 1$. If this condition is violated, the difference vector is normalized and scaled to unit length, yielding the final commanded velocity:

$$\mathbf{v}_i^{\text{u-des-2}}(t) = \mathbf{v}_i^{\text{true}}(t) + \frac{d\mathbf{v}_i(t)}{\|d\mathbf{v}_i(t)\|_2}. \quad (11)$$

This effectively caps the instantaneous change in velocity to a magnitude of 1 m/s, ensuring smooth transitions in the

commanded trajectory.

Finally, we impose an angular constraint to prevent excessively sharp turns. Specifically, we limit the angle between the current velocity $\mathbf{v}_i^{\text{true}}(t)$ and the adjusted desired velocity $\mathbf{v}_i^{\text{u-des-2}}(t)$ to at most $\pi/2$ radians. Let the angle between these vectors be defined as

$$\theta_i(t) = \angle(\mathbf{v}_i^{\text{u-des-2}}(t), \mathbf{v}_i^{\text{true}}(t)). \quad (12)$$

If $\theta_i(t) > \pi/2$, we adjust the desired velocity to lie exactly at the maximum allowed angle. This is achieved by projecting $\mathbf{v}_i^{\text{u-des-2}}(t)$ onto the plane orthogonal to $\mathbf{v}_i^{\text{true}}(t)$ and using this orthogonal component as the new desired direction. Define the unit vector in the direction of the current velocity:

$$\mathbf{e}_{1,i}(t) = \frac{\mathbf{v}_i^{\text{true}}(t)}{\|\mathbf{v}_i^{\text{true}}(t)\|_2}. \quad (13)$$

The component of $\mathbf{v}_i^{\text{u-des-2}}(t)$ orthogonal to $\mathbf{v}_i^{\text{true}}(t)$ is then obtained via

$$\mathbf{e}_{2,i}(t) = \mathbf{v}_i^{\text{u-des-2}}(t) - \langle \mathbf{v}_i^{\text{u-des-2}}(t), \mathbf{e}_{1,i}(t) \rangle \mathbf{e}_{1,i}(t), \quad (14)$$

where $\langle \cdot, \cdot \rangle$ denotes the standard inner product. The final, angle-limited velocity command in the world frame is set to this normalized orthogonal component:

$$\mathbf{v}_i^{\text{u-des-3}}(t) = \frac{\mathbf{e}_{2,i}(t)}{\|\mathbf{e}_{2,i}(t)\|_2}. \quad (15)$$

This constrains the desired velocity to be exactly perpendicular to the current velocity when the original desired direction would have exceeded the $\pi/2$ threshold.

After applying all constraints, the world-frame velocity command $\mathbf{v}_i^{\text{u-des-3}}(t)$ is transformed into the body frame, denoted as $\mathbf{v}_i^{\text{b}}(t)$ for use by the low-level controller.

Yaw Rate Control. The yaw rate is computed using a proportional controller based on the heading error. The desired heading $\psi_{d,i}(t)$ is derived from the desired body-frame velocity vector:

$$\psi_{d,i}(t) = \arctan\left(\frac{(\mathbf{v}_i^{\text{b}}(t))_y}{(\mathbf{v}_i^{\text{b}}(t))_x}\right). \quad (16)$$

Assuming the current yaw angle is zero (or the error is computed relative to the current orientation), the yaw error is $\psi_{e,i}(t) = \psi_{d,i}(t)$. The yaw rate command is then:

$$\dot{\psi}_i(t) = \gamma \psi_{e,i}(t), \quad \gamma > 0, \quad (17)$$

where γ is the proportional gain.

Bringing It All Together. After computing the saturated body-frame linear velocity $\mathbf{v}_i^{\text{b}}(t)$ and the yaw rate $\dot{\psi}_i(t)$, the final velocity command vector for agent i is:

$$\mathbf{u}_i(t) = \begin{pmatrix} \mathbf{v}_i^{\text{b}}(t) \\ \dot{\psi}_i(t) \end{pmatrix}, \quad (18)$$

which serves as the input to the low-level flight controller.

IV. SYSTEM ARCHITECTURE AND IMPLEMENTATION

This section outlines the software architecture and implementation details of the simulation framework. The framework is built on three core components: Docker for containerized, reproducible environments; Webots for physics-based robotics simulation; and ROS 2 for distributed communication between agents and control nodes. All source code can be found at [6].

A. Docker

To ensure reproducibility and simplify deployment, the environment is containerized using Docker. The Docker image has been based off of the work of [8]. The version we have installed is Webots R2025a. We've also installed ROS 2 Humble, together with webots-ros2, which includes the Mavic flight controller, and the interface between ROS 2 and Webots.

B. Webots

Webots simulates the rigid-body dynamics of the swarm within an obstacle-free, flat-ground arena in $\mathbb{R}^3$, using its default physics model with a global damping coefficient of 0.5 for numerical stability.

The Quadcopters (DJI Mavic PRO models) are spawned via a launch file into a world defined by a Webots world file (see [6]), each initialized at a fixed position with zero velocity. Control logic is executed externally through ROS 2 nodes, using Webots' external controller interface (`<extern>`) so that Webots handles only physical consistency while high-level control and communication reside outside the simulator.

Environmental disturbances, sensor noise, actuator failures and so on are omitted, yielding an idealized simulation with perfect sensing and actuation; results thus reflect nominal operating conditions.

C. ROS 2

ROS2 serves as the software framework for communication among distributed drone nodes in simulation time. While controllers can be written directly in Webots using C++, MATLAB, Python, or other languages, ROS2 offers a structured and scalable communication layer that simplifies data exchange between multiple agents and the target.

Each quadcopter is launched as an independent ROS2 node. These nodes publish their local states—such as global position, velocity, and orientation to designated ROS2 topics. Every agent subscribes to the state topics of all other drones, enabling each controller to incorporate inter-agent information when computing control inputs. This topic based communication constitutes the sole means of inter-agent interaction and ensures a decentralized control architecture.

The target is implemented as a separate ROS2 node representing a point in space. Though static and without intrinsic dynamics, its position is defined within the Webots world and

published as a global coordinate. All drone nodes subscribe to this target topic, allowing them to update their guidance accordingly. The target node also subscribes to the global positions of the quadcopters, triggering a respawn when a drone enters a predefined radius.

V. EXPERIMENTAL RESULTS

The experiment consist of simulating three DJI Mavic 2 Pro quadcopters equipped with the proposed controller in the Webots-ROS2 framework described earlier. The simulations are executed while recording time-stamped logs. Each log entry contains agent states (positions and velocities), control inputs, and discrete events (target spawning and capture). The raw text logs are parsed offline and transformed into the tables and figures presented in this section.

An *episode* is defined as the time interval between the spawning of a target and the instant at which it is reached by any agent.

A drone that actively seeks the target, is classified as active, and drones that are not are inactive. A drone may become inactive as a result of a collision with another drone or due to loss of stability leading to uncontrolled motion.

A. Target reaching

The data presented here represents the time efficiency of each quadcopter, and are summarized in a table where the columns correspond to individual quadcopters, and the row index denotes the target being reached. As this target is respawning, the index is ordered chronologically, such that target 1 is the first episode, and target 6 is the last episode. The symbol “–” indicates that drone i became inactive during the corresponding episode and therefore did not produce a valid measurement.

TABLE I
AVERAGE LINEAR VELOCITY (M/S) FOR EACH DRONE AND FLAG EPISODE

Target	Drone 1	Drone 2	Drone 3
1	0.6681	0.6704	0.7384
2	0.9552	0.9275	–
3	0.9034	0.9209	–
4	0.9622	0.9637	–
5	0.8597	0.8862	–
6	0.918	–	–

B. Collision avoidance

To examine inter-agent proximity, the pairwise Euclidean distance between drones i and j are plotted over time, as shown below.

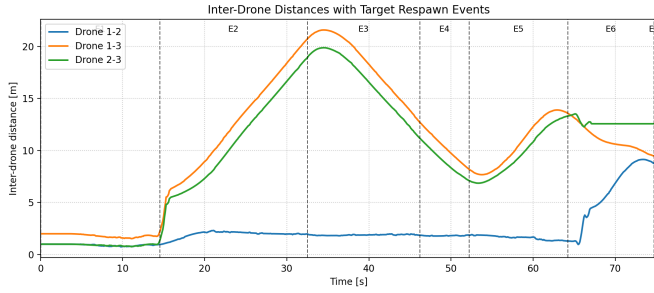


Fig. 1. Pairwise distance between drones

On the horizontal-axis, the time in seconds are plotted, and the distance in meters are plotted on the vertical axis. The vertical dotted gray lines indicate the separation between episodes.

C. Overview

As an overview of the whole simulation, we have displayed a plot showing the Euclidean distance between the agents and the target.

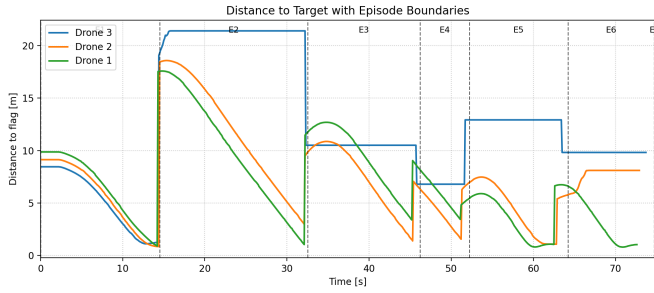


Fig. 2. Distance between drones and target

The plot spans all target episodes in chronological order. Discontinuities in the distance curves coincide with target respawn events, after which new distance trajectories are initiated. Intervals in which $d_i(t)$ remains constant correspond to agents that do not change position during that episode time interval since they've become inactive.

VI. DISCUSSION

A. Target reaching

Figure 2 shows that all three drones successfully reach the first target. After the completion of the first episode, drone 3 becomes inactive. This is reflected by the constant distance between drone 3 and the target for episodes two through six, indicating that the drone is no longer responding to published velocity commands.

During episode two, drones 1 and 2 show monotone convergence towards the target, demonstrating successful target-reaching behavior. Table I also shows that both drones maintain velocities close to the saturation limits, indicating relatively high speeds during the approach. This pattern

continues across subsequent episodes until episode five, after which the distance between drone 2 and the target also become constant. At this point, drone 2 becomes inactive. Once only a single drone remains, the simulation is terminated, since the remaining drone can reach all subsequent targets without interaction from other agents.

B. Collision avoidance

From Fig. 1 it is evident that the distances between Drone 1 and Drone 3, and between Drone 2 and Drone 3, increase sharply after the first episode. This occurs because Drone 3 becomes inactive at that point, as discussed previously. In contrast, Drone 1 and Drone 2 maintain a relatively small but nonzero separation, indicating coordinated motion without collision. This behaviour suggests that the repulsive component of the APF-based controller is effective.

After Episode 5, the distance between Drone 1 and Drone 2 increases markedly as Drone 2 also becomes inactive. This transition is further reflected in the approximately constant distance between Drone 2 and Drone 3 thereafter, since both agents remain stationary.

C. Unstable flight controller?

During simulation we frequently observed that the Mav-ics became inactive (i.e. lost stability and crashed) : small increases in roll produced controller overcompensation that flipped the vehicle. We initially hypothesized that velocity commands were too aggressive and therefore applied saturation limits, which reduced but did not eliminate failures. Inspection of the underlying controller implementation indicated unstable attitude-control as the primary cause. No suitable open-source replacement controller was found, and implementing a bespoke low-level motor controller was outside the project scope. Consequently, these controller instabilities should be borne in mind when interpreting the simulation results.

VII. CONCLUSION AND FUTURE WORK

This work investigated whether an artificial potential field based controller can enable a quadcopter swarm to repeatedly reach sequential targets while maintaining collision avoidance in the absence of centralized coordination.

Simulation results under idealized conditions show that the proposed decentralized strategy achieves these objectives. Active agents reliably converge toward each target, while the repulsive interaction terms maintain safe inter-agent separations throughout the mission. The system continues to function even as individual drones become inactive, with the remaining agents successfully acquiring subsequent targets.

All coordination emerges from decentralized, per-agent computations based solely on shared state information communicated via ROS2, without any centralized trajectory planning or task allocation. While the results validate the feasibility of

APF-based decentralized control for repeated target-reaching in simulation, future work should focus on improving low-level flight stability, incorporating realistic disturbances and sensing constraints, and evaluating the approach on physical platforms to assess robustness beyond idealized simulation settings.

A. Future work

Several limitations and extensions motivate future research. First, in scenarios where two drones face each-other head-on with parallel velocity vectors, the current repulsion force may cause both agents to oscillate and retreat back and forth in a deadlock-like state. Incorporating additional coordination strategies such as explicit lateral avoidance rules or short-horizon trajectory negotiation could resolve such situations, by breaking symmetry and enabling bypass maneuvers.

Second, the repulsion gains are static, and not optimized. This limits performance in dense swarm formations, and could lead to crashes for the drones located closest to the center of the swarm formation. Future work should investigate adaptive or optimized gain selection strategies, where repulsion strengths vary as a function of local agent density.

There is also significant potential to view the proposed controller through the lens of mechanics and dynamic systems theory. Since the repulsion and attraction vector fields have positive and negative divergence, and the velocity vector being publishes is the sum of these, then we can involve the theory from mechanics, and possibly make new discoveries.

REFERENCES

- [1] W. Ren and R. Beard, *Distributed Consensus in Multi-vehicle Cooperative Control: Theory and Applications*, ser. Communications and Control Engineering. Springer London, 2007. [Online]. Available: <https://books.google.no/books?id=e2jXXWxWjmMC>
- [2] A. Waga, S. Benhlila, A. Bekri, J. Abdouni, and F. Z. Saber, "A survey on autonomous navigation for mobile robots: From traditional techniques to deep learning and large language models," *Journal of King Saud University Computer and Information Sciences*, vol. 37, pp. 20–23, 08 2025.
- [3] O. Michel, "Webotstm: Professional mobile robot simulation," *International Journal of Advanced Robotic Systems*, vol. 1, no. 1, pp. 40–43, 2004.
- [4] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022. [Online]. Available: <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>
- [5] Cyberbotics Ltd., "webots_ros2: Official ros 2 packages for webots robot simulator," 2025, accessed: 2025-12-18. [Online]. Available: https://github.com/cyberbotics/webots_ros2
- [6] O. Ekeberg and D. Grønhaug, "DroneSwarm_project_TEK4090: Drone Swarm Simulation with APF Controller," https://github.com/OliverOE1509/DroneSwarm_project_TEK4090, 2025, accessed: 2025-12-18.
- [7] P. Barroso, "drone-simulation," <https://github.com/patrickpbarroso/drone-simulation>.
- [8] Cyberbotics, "webots-docker: Docker images for Webots robot simulator," <https://github.com/cyberbotics/webots-docker>, 2025, accessed: 2025-12-18.
- [9] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *The International Journal of Robotics Research*, vol. 5, pp. 90 – 98, 1985. [Online]. Available: <https://api.semanticscholar.org/CorpusID:31541613>